Kantonsschule Im Lee Winterthur

Maturitätsarbeit HS 2026/27

Zugängliche Informationen trotz Verschlüsselung, die Macht der Metadaten.

Bernard Rognon, Tim Frauenfelder 4f

Betreut durch: Thomas Graf

Winterthur

18. Oktober 2026

Inhaltsverzeichnis

1 Einleitung **2**

2 Motivation **3**

3 Client-Server-Struktur **4**
3.1 Funktionsprinzip der Client-Server-Struktur 4
3.1.1 Der Aufbau 5
3.1.2 Verbindung zwischen Client und Server 5
3.1.3 Kryptologie 5
3.2 Vergleich mit anderen Strukturen 6

4 Die Metadaten **7**

5 Unser Programm **8**
5.1 Architektur und Systemübersicht 8
5.1.1 Das Client-Server-Modell 8
5.1.2 Kommunikationsprotokoll 8
5.1.3 Datenfluss 8
5.2 Server-Hardware und Betriebsumgebung 9
5.2.1 Hardware-Basis 9
5.2.2 Ressourcenbelastung und Netzwerkanbindung 9
5.2.3 Betriebssystem und Programmiersprache 9
5.3 Server-Side: Das Backend 9
5.3.1 Netzwerk und Verbindungsmanagement 9
5.3.2 Datenpersistenz und Datenbank 9
5.3.3 Sicherheit und Authentifizierung auf Serverseite 9
5.4 Clientside: Benutzeroberfläche und Client-Logik 9
5.4.1 Verbindungsaufbau 9
5.4.2 Asynchrone Nachrichtenverarbeitung 9
5.4.3 Kryptographie und Datenintegrität 9
5.5 Zusammenfassung 9

6 Metadaten in unserem Programm **10**

7 Reflexion **11**

8 Future Work **12**

9 Fazit **13**

Abbildungsverzeichnis **14**

Quellenverzeichnis **15**

Kapitel 1

Einleitung

Kapitel 2

Motivation

Im September 2025 erschienen plötzlich auf vielen sozialen Medien und News-Plattformen Beiträge, die sich mit der Child Sexual Abuse Regulation (CSAR) befassten. Die CSAR, auch bekannt als ”EU Chat Control 2.0”, ist ein Gesetzentwurf der EU, dessen Ziel es ist, das Verteilen von Kinderpornographie einzudämmen und Kinder vor der Anbahnung sexueller Kontakte im Netz zu schützen. Dazu sollte in allen EU-Staaten jede einzelne versendete private Nachricht und Datei vor dem Versenden von einer KI geprüft werden. Viele EU-Bürger:innen machten sich Sorgen einen Teil ihrer Privatsphäre zu verlieren, da das Programm die Ende-zu-Ende-Verschlüsselung umging und eine permanente Überwachung aller EU-Bürger:innen bedeutete. Ausserdem würde es unabhängigen Staaten erleichtern, eigene Gesetze zur Überwachung der Online-Kommunikation zu erlassen.
Obwohl wir nicht direkt betroffen waren von der CSAR, da die Schweiz nicht Teil der EU ist, haben wir uns mit dem Thema Online-Überwachung auseinandergesetzt und uns dazu ausgetauscht. Dabei kamen wir zum Schluss: Wir wollen ein eigenes Messaging-Programm schreiben und hosten, welches nicht von Drittpersonen überwacht werden kann. Kurz darauf machten wir uns auf die Suche nach einem alten PC, welchen wir zu einem Linux-Homeserver umbauen konnten. Auf Ricardo fanden wir einen Lenovo Office-PC für 50 CHF, welcher sich perfekt für unser Projekt eignete. Im selben Monat fuhren wir nach Rüti ZH, um ihn abzuholen.

Anfang Januar hatten wir noch keine Zeile Code geschrieben. Wir hatten auch beide keine Ahnung, zu welchem Thema wir unsere Maturitätsarbeit schreiben wollen. Einem von uns kam die Idee, diese zwei Projekte zu kombinieren. Wir fragten Thomas Graf, ob es möglich ist, die Maturitätsarbeit zu zweit zu schreiben und ob das Programmieren eines Messaging-Programms ein geeignetes Thema wäre. Als er Ja sagte, war für uns beide klar: Dieses Projekt wird zu unserer Maturitätsarbeit. Wir wollen zusammen in Python ein möglichst sicheres Messaging-Programm schreiben. Dieses werden wir dann auf unserem Server hosten. Was uns noch fehlte, war ein Schwerpunkt für die Arbeit. Auf einer langen Zugfahrt nach Interlaken sammelten wir viele Ideen, die wir sofort wieder verworfen haben; der Grund dafür war meistens, dass sie zu viel Mathematik involvierten. Im Endeffekt entschieden wir uns, uns damit zu beschäftigen, wie wir die Menge an sammelbaren Metadaten minimieren oder sogar komplett eliminieren können. Dies ist ein sehr aktuelles Thema, da Metadaten zu sammeln die einfachste Art ist, trotz einer Ende-zu-Ende-Verschlüsselung, private Informationen über die Nutzer eines Messaging-Programms zu sammeln.

Kapitel 3

Client-Server-Struktur

Das Ziel eines Messaging-Programmes ist es, von Gerät A eine Nachricht an Gerät B zu versenden. Dabei sollte möglichst wenig Rechenleistung beansprucht werden. Das Versenden soll schnell erfolgen, und die Nachricht darf für niemanden ausser den Absender und den Empfänger lesbar sein. Zum Versenden von Nachrichten gibt es schon sehr viele Programme. Die meisten können anhand ihres Funktionsprinzips in drei Unterkategorien eingeteilt werden: Diese sind die Client-Server-Struktur, das Peer-to-Peer-Messaging und das föderierte Servermodell. Die Client-Server-Struktur ist die verbreitetste Architektur für Messaging-Programme und auch diejenige, welche wir für unser Projekt benutzen. Deshalb werden wir in diesem Kapitel die Theorie dahinter erklären.

3.1 Funktionsprinzip der Client-Server-Struktur

Bekannte Beispiele für Programme, die die Client-Server-Struktur verwenden, sind WhatsApp, Signal und Discord. Dabei werden Clients benutzt, die sich alle mit einem Server verbinden [1]. Der Client ist ein Programm, welches auf dem Gerät jedes Nutzers läuft. Er dient dazu, Nachrichten an den Server zu senden und von ihm zu empfangen. Dazu hat er ein Interface, auf dem die Nachrichten angezeigt werden und neue verfasst werden können. Der Server ist eine zentrale Rechenmaschine, welche gleichzeitig mit allen Clients verbunden ist. Er speichert die vom Client verschickten Nachrichten ab, bis er eine Verbindung zum gewählten Empfänger hergestellt hat, und leitet sie dann an ihn weiter. Zusätzlich kann der Empfänger, wenn er die Nachricht empfängt und liest, dies dem Server mitteilen, und der Server leitet diese Information an den Sender zurück. Einige Programme, wie zum Beispiel Discord, löschen die Nachrichten nicht vom Server. Wenn dies gemacht wird kann der Nutzer alle vergangenen Nachrichten auf einem neuen Gerät abrufen.

[1] Dies kann auch einen Server Cluster

3.1.1 Der Aufbau

3.1.2 Verbindung zwischen Client und Server

3.1.3 Kryptologie

Damit von einem Messaging-Programm Privatsphäre gewährleistet wird, darf niemand ausser Absender und Empfänger Zugriff auf die versendete Nachricht haben. Dazu wird ein Verfahren namens Verschlüsselung verwendet. Wenn Nachrichten ohne Verschlüsselung verschickt werden, erlaubt das jedem, der Zugriff auf das Netzwerk der beteiligten Clients oder des Servers hat, die Nachricht auch zu lesen.

Die Definition von Wikipedia:

> *"Verschlüsselung nennt man die Umwandlung von Informationen, genannt Klartext, in einen Geheimtext. Dies geschieht mithilfe von kryptografischen Verfahren unter Verwendung eines geheimzuhaltenden Schlüssels, der nur den befugten Personen bekannt sein darf. Mit Kenntnis des Schlüssels kann der Geheimtext wieder in den Klartext zurückgewandelt werden, was Entschlüsselung genannt wird."*

Bei einer Ende-zu-Ende-Verschlüsselung wird garantiert, dass niemand ausser dem Absender und dem Empfänger die Nachricht lesen kann. Dabei wird die Nachricht am Ende des Absenders verschlüsselt und kann nur am Ende des Empfängers wieder entschlüsselt werden. Niemand dazwischen kann die Nachricht entschlüsseln. Der Server erhält also nur eine verschlüsselte Version der Nachricht. Um die verschlüsselte Nachricht wieder zu entschlüsseln, braucht der Empfänger jedoch zwingend den korrekten Schlüssel. Dies ist bei einer herkömmlichen symmetrischen Verschlüsselung, derselbe Schlüssel wie zur Verschlüsselung verwendet wurde. Somit entsteht das Problem, dass er dem Empfänger mitgeteilt werden muss. Durch das Versenden des Schlüssels hätte jedoch jeder Zugriff auf den Schlüssel und die verschlüsselte Nachricht. Die Nachricht könnte dann von allen entschlüsselt werden. Um dieses Problem zu lösen, wird eine asymmetrische Verschlüsselung benutzt.

asymmetrische Verschlüsselung

Die asymmetrische Verschlüsselung zeichnet sich dadurch aus, dass sie nicht den gleichen Schlüssel zur Verschlüsselung und Entschlüsselung verwendet. Dafür gibt es verschiedene Algorithmen, die alle dem gleichen Prinzip folgen. Bei der Erstellung eines Accounts generiert der Client A ein sogenanntes Key-Pair. Dieses besteht aus einem Public- und einem Private-Key. Der Public Key wird offen an alle anderen Nutzer versendet, der Private Key wird lediglich auf dem Gerät des Nutzers A gespeichert. Wenn Client B eine Nachricht an Client A versenden möchte, benutzt er den Public Key von Client A, um diese zu verschlüsseln. Der Public Key kann aber nicht verwendet werden, um die Nachricht wieder zu entschlüsseln. Eine Entschlüsselung ist nur mit dem Private Key von Client A möglich. So wird eine sichere Kommunikation ermöglicht, ohne die Schwachstelle, dass ein Schlüssel zur Entschlüsselung ausgetauscht werden muss. Solange alle Private Keys geheim bleiben, ist völlige Privatsphäre gewährleistet. Der Nachteil an diesem Verfahren ist, dass die Algorithmen für

die asymmetrische Verschlüsselung viel langsamer sind und mehr Rechenleistung beanspruchen.

Hybrid-Kryptosystem

Die wichtigste Art der Verschlüsselung ist das Hybrid-Kryptosystem. Es ist, wie der Name schon andeutet, eine Kombination aus der asymmetrischen und der symmetrischen Verschlüsselung. Während dem Herstellen einer Verbindung zwischen Client A und Server wird von Client A ein zufälliger Session Key generiert. Dieser wird dann benutzt, um die Nachricht, welche den Grossteil der Datenmenge ausmacht, effizient symmetrisch zu verschlüsseln. Danach wird der Session Key asymmetrisch verschlüsselt mit dem Public Key von Client B. Da der Session Key sehr klein ist, wird trotz asymmetrischer Verschlüsselung wenig Zeit und Rechenleistung beansprucht. Danach werden die verschlüsselte Nachricht und der verschlüsselte Session Key an Client B versendet. Dieser entschlüsselt zuerst den Session Key mit seinem Private Key und kann dann die Nachricht mit dem Session Key symmetrisch entschlüsseln. Bei diesem Vorgang wird die grosse Datenmenge effizient verschlüsselt mit der symmetrischen Verschlüsselung und ist trotzdem sicher, da der Session Key asymmetrisch verschlüsselt wird. Das Hybrid-Kryptosystem ist der heutige Standard und wird unter anderem für HTTPS gebraucht, das Protokoll das hauptsächlich genutzt wird, um im Internet Daten zu übertragen.

3.2 Vergleich mit anderen Strukturen

Kapitel 4

Die Metadaten

Kapitel 5

Unser Programm

5.1 Architektur und Systemübersicht

5.1.1 Das Client-Server-Modell

Für unseren Ansatz eines Messengers haben wir uns aus praktischen Gründen für ein Client-Server-Modell entschieden. Da die meisten modernen Messenger ebenfalls so funktionieren hatten wir somit Zugang zu vielen Informationen von bereits funktionierenden Systemen und konnten schon bestehende Hardware weiter nutzen.

5.1.2 Kommunikationsprotokoll

Für die Kommunikation zwischen Server und Client sind TCP-Sockets zuständig. Diese TCP-Sockets erstellen eine Verbindung zwischen den beiden Geräten mithilfe von IP-Adressen und Ports. Das zugrundeliegende Kommunikationsprotokoll TCP (Transmission Control Protocol) nutzt einen lokalen TCB (Transmission Control Block). In diesem sind wichtige Informationen gespeichert, unter anderem die IP-Adresse und der Port des Partner Sockets, den Verbindungsstatus (z. B. verbunden, wartend, geschlossen), welche Daten noch gesendet oder gelesen werden müssen und die Sequence Numbers, welche die Reihenfolge der empfangenen und versendeten Bytes festhalten. Dieser Aspekt ist für unsere Anwendung entscheidend: Durch den Abgleich der Sequence Numbers und Bestätigungspakete garantiert TCP, dass die Daten vollständig, fehlerfrei und in der korrekten Abfolge ankommen.

5.1.3 Datenfluss

Sobald jemand eine Nachricht versendet, erstellt der Client eine TCP-Verbindung zum Server und authentifiziert sich. Dann wird die Nachricht noch lokal verschlüsselt und in die eigens entwickelte Nachrichtenstruktur eingebunden. In dieser werden Statuscodes genutzt, um Nachrichten nach Zweck zu kategorisieren (siehe Tabelle 5.1). Je nach Kategorie enthält das Paket dann noch zusätzliche Metadaten wie zum Beispiel den Zeitstempel oder den Session-Key zur Verifikation der Sitzung. Erst jetzt wird dieses ganze Paket an den Server versendet. Dort wird es entpackt und in die einzelnen Bestandteile aufgeteilt. Da es sich in diesem Beispiel um eine

Nachricht an einen anderen Benutzer handelt, überprüft der Server die momentane Erreichbarkeit des Empfängers. Falls dieser erreichbar sein sollte, verpackt er die Nachricht wieder mit den notwendigen Informationen und leitet sie weiter. Falls der Empfänger nicht erreichbar sein sollte, wird die Nachricht vorerst auf dem Server gespeichert, bis der Empfänger sich das nächste Mal mit dem Server verbindet, woraufhin die Nachricht zugestellt wird.

Tabelle 5.1: Übersicht der anwendungsspezifischen Protokoll-Codes

Code	Kategorie / Zweck	Erforderliche Metadaten / Payload
0	Authentifizierung	Benutzername, Passwort-Hash, Timestamp
100	Server-Status	Session-Key, Status-Flag (z. B. Ping/Keep-Alive)
200	Benutzernachricht	Empfänger-ID, Session-Key, verschlüsselte Nachricht

5.2 Server-Hardware und Betriebsumgebung

5.2.1 Hardware-Basis

Als Grundlage für unseren Server dient ein Lenovo ThinkCentre mit einem Intel i7-7700 und 8 GB Arbeitsspeicher. Dies ist mehr als genug Rechenleistung, um mehrere Benutzer gleichzeitig zu bedienen. Dazu kommt, dass ein solches System für die zu erwartende Last viel effizienter ist.

5.2.2 Ressourcenbelastung und Netzwerkanbindung

5.2.3 Betriebssystem und Programmiersprache

5.3 Server-Side: Das Backend

5.3.1 Netzwerk und Verbindungsmanagement

5.3.2 Datenpersistenz und Datenbank

5.3.3 Sicherheit und Authentifizierung auf Serverseite

5.4 Clientside: Benutzeroberfläche und Client-Logik

5.4.1 Verbindungsaufbau

5.4.2 Asynchrone Nachrichtenverarbeitung

5.4.3 Kryptographie und Datenintegrität

5.5 Zusammenfassung

Kapitel 6

Metadaten in unserem Programm

Kapitel 7

Reflexion

Kapitel 8

Future Work

Kapitel 9

Fazit

Abbildungsverzeichnis

Quellenverzeichnis

[1] fhdrsdg. *How to expand a frame inside a canvas in tkinter*. 2018. URL: https://stackoverflow.com/questions/50243679/how-to-expand-a-frame-inside-a-canvas-in-tkinter (besucht am 10. 10. 2026).

[2] GeeksforGeeks. *Scrollable Frames in Tkinter*. 2026. URL: https://www.geeksforgeeks.org/python/scrollable-frames-in-tkinter/ (besucht am 10. 10. 2026).

[3] Peter Grill. *When to use \par and when \newline or blank lines?* URL: https://tex.stackexchange.com/questions/82664/when-to-use-par-and-when-newline-or-blank-lines (besucht am 13. 10. 2026).

[4] Harshith. *Deep Dive into TCP Sockets: How Data Travels Under the Hood*. 2025. URL: https://medium.com/@harshithgowdakt/deep-dive-into-tcp-sockets-how-data-travels-under-the-hood-7c16f6b2bf95 (besucht am 14. 10. 2026).

[5] Heise. *Creating a list of figures*. URL: https://www.heise.de/tipps-tricks/LaTeX-Abbildungsverzeichnis-automatisch-erstellen-7261837.html (besucht am 13. 10. 2026).

[6] Freddy Mcloughlan. *Implementing biber for biblatex in Microsoft Visual Studio Code*. 2024. URL: https://tex.stackexchange.com/questions/459640/implementing-biber-for-biblatex-in-microsoft-visual-studio-code (besucht am 13. 10. 2026).

[7] Overleaf. *Creating a bibliography*. URL: https://docs.overleaf.com/citing-and-references/creating-a-bibliography (besucht am 13. 10. 2026).

[8] Overleaf. *How do I adjust the font size?* URL: https://www.overleaf.com/learn/latex/Questions/How_do_I_adjust_the_font_size%3F (besucht am 13. 10. 2026).

[9] Overleaf. *Learn LaTeX in 30 minutes*. URL: https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes (besucht am 12. 10. 2026).

[10] Overleaf. *Page size and margins*. URL: https://www.overleaf.com/learn/latex/Page_size_and_margins (besucht am 13. 10. 2026).

[11] ron rothman. *Converting epoch time into the datetime*. 2023. URL: https://stackoverflow.com/questions/12400256/converting-epoch-time-into-the-datetime (besucht am 09. 10. 2026).

[12] TKDocs. *Tkinter Tutorial*. URL: https://tkdocs.com/tutorial/ (besucht am 23. 09. 2026).

[13] W3Schools. *Python Global Variables*. URL: https://www.w3schools.com/python/python_variables_global.asp (besucht am 07. 10. 2026).

[14] W3Schools. *Python Tuples*. URL: `https://www.w3schools.com/python/python_tuples.asp` (besucht am 04. 10. 2026).